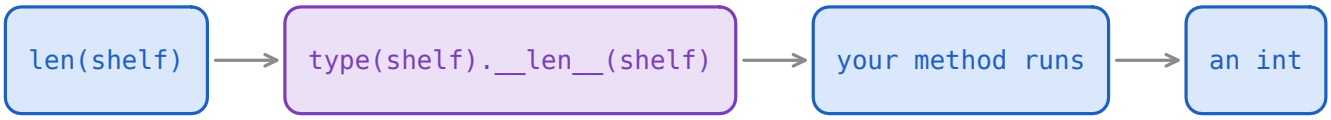
Chapter 5

Classes and the data model

every piece of syntax is a method call you can define

You've written classes with `__init__` and methods. The step up is the **data model**: Python turns almost every piece of syntax into a call to a specially named ("dunder") method. Define the method, and your objects work with the syntax, the built-ins and the whole standard library.



you write	Python calls	you write	Python calls
`a + b`	`a.__add__(b)`	`len(x)`	`x.__len__()`
`a == b`	`a.__eq__(b)`	`x[k]`	`x.__getitem__(k)`
`a < b`	`a.__lt__(b)`	`k in x`	`x.__contains__(k)`
`for v in x`	`iter(x)` → `x.__iter__()`	`bool(x)`, `if x:`	`x.__bool__()` or `__len__`
`repr(x)`	`x.__repr__()`	`f(...)` on an object	`x.__call__(...)`
`hash(x)`	`x.__hash__()`	`with x:`	`__enter__`, `__exit__` (Ch 6)

KEY IDEA

This is why `len` is a function and not a method: `len(x)` works on any object whose class defines `__len__`. It's an interface, not inheritance. Ask "which protocol does this syntax use?" and you can make your own types feel built in.

Where attributes live

An instance has its own dict of attributes. Methods live on the **class**. `obj.name` looks in the instance first, then the class, then its base classes in order:

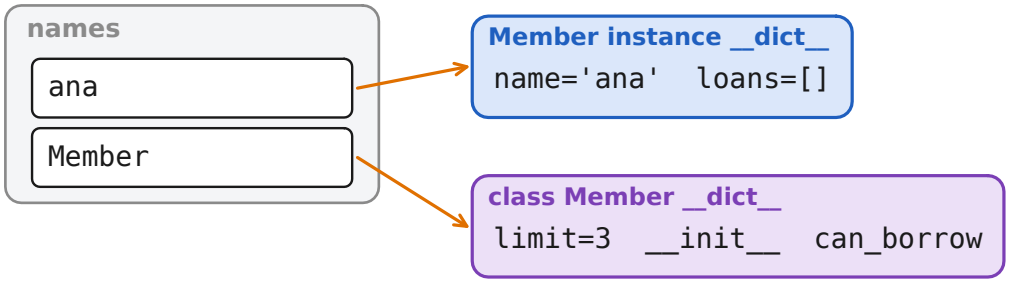
```
class Member:
    limit = 3                          # class attribute: shared by all members

    def __init__(self, name):
        self.name = name               # instance attribute
        self.loans = []

    def can_borrow(self):
        return len(self.loans) < self.limit

ana = Member("ana")
print(vars(ana), "limit" in vars(ana), ana.limit)
print(ana.can_borrow(), Member.can_borrow(ana))     # the same call
```

```
{'name': 'ana', 'loans': []} False 3
True True
```



`ana.can_borrow()` finds the function on the class and passes `ana` as `self`. That's all a method is: a function stored on the class, with the instance filled in as its first argument.

```
ana.limit = 10                  # creates an INSTANCE attribute that hides the class one
ben = Member("ben")
print(ana.limit, ben.limit, Member.limit)
```

```
10 3 3
```

TRAP

A mutable **class** attribute (`loans = []` at class level) is shared by every instance, the same bug as the mutable default argument. Create per-object state in `__init__`.

repr, equality and hashing

A class you write gets the default `repr` (`<Book object at 0x7f...>`) and identity equality (two books with the same title are not `==`). For a **value** type you want both fixed:

```python
class Book:
    def __init__(self, title, year):
        self.title, self.year = title, year

    def __repr__(self):
        return f"Book({self.title!r}, {self.year})"     # looks like the code to rebuild it

    def __eq__(self, other):
        if not isinstance(other, Book):
            return NotImplemented                       # let the other side try
        return (self.title, self.year) == (other.title, other.year)

    def __hash__(self):
        return hash((self.title, self.year))            # equal objects MUST hash equal

a, b = Book("Dune", 1965), Book("Dune", 1965)
print(a, a == b, a is b, len({a, b}))
```

```
Book('Dune', 1965) True False 1
```

TRAP

Defining `__eq__` without `__hash__` makes instances **unhashable** (Python sets `__hash__ = None` to protect you). And only hash fields that never change; mutate a key after putting it in a set and the set can't find it again.

dataclasses write all of that for you

by hand

```python
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __repr__(self):
        x, y = self.x, self.y
        return f"Point({x=}, {y=})"
    def __eq__(self, o):
        return (self.x, self.y) == (o.x, o.y)
    def __hash__(self):
        return hash((self.x, self.y))
print(Point(1, 2) == Point(1, 2), Point(1, 2))
```

```
True Point(x=1, y=2)
```

@dataclass(frozen=True)

```python
from dataclasses import dataclass

@dataclass(frozen=True)
class Point:
    x: int
    y: int



print(Point(1, 2) == Point(1, 2), Point(1, 2))
```

```
True Point(x=1, y=2)
```



dataclass option	gives you
`@dataclass`	`__init__`, `__repr__`, `__eq__` from the annotated fields
`frozen=True`	immutable fields (assignment raises), and `__hash__`
`order=True`	`<`, `<=`, `>`, `>=` comparing fields in order (so `sorted` works)
`field(default_factory=list)`	a fresh mutable default per instance (no shared-list bug)
`slots=True`	`__slots__`: smaller instances, no accidental new attributes

Container and iteration protocols

Implement a few dunders and your object works with `len`, `in`, indexing, slicing, `for`, `sorted`, `list()`, unpacking, `sum`... none of which knows your class exists:

```
1  class Shelf:
2      def __init__(self, titles=()):
3          self._titles = list(titles)
4
5      def __len__(self):
6          return len(self._titles)
7
8      def __getitem__(self, i):                    # handles ints AND slices
9          got = self._titles[i]
10         return Shelf(got) if isinstance(i, slice) else got
11
12     def __iter__(self):
13         yield from self._titles
14
15     def __contains__(self, title):
16         return title.lower() in (t.lower() for t in self._titles)
17
18     def __repr__(self):
19         return f"Shelf({self._titles})"
20
21 s = Shelf(["Dune", "Emma", "Ubik"])
22 print(len(s), s[0], s[1:], "emma" in s, sorted(s, reverse=True))
23 first, *rest = s
24 print(first, rest, bool(Shelf()))
```

```
3 Dune Shelf(['Emma', 'Ubik']) True ['Ubik', 'Emma', 'Dune']
Dune ['Emma', 'Ubik'] False
```

line 9: a slice returns a Shelf

line 15: case-insensitive

WHY

`bool(Shelf())` is `False` without a `__bool__` method: Python falls back to `__len__`, and zero length means false. Protocols have sensible fallbacks: with only `__getitem__`, `for` and `in` still work by trying indexes 0, 1, 2... until `IndexError`.

Operators: a money type

Arithmetic operators are two-sided. For `a + b` Python tries `a.__add__(b)`; if that returns `NotImplemented`, it tries `b.__radd__(a)`. That's what makes `sum()` (which starts from `0`) work:

```python
from dataclasses import dataclass
from decimal import Decimal
from functools import total_ordering

@total_ordering
@dataclass(frozen=True)
class Money:
    pence: int

    def __add__(self, other):
        if isinstance(other, Money):
            return Money(self.pence + other.pence)
        return NotImplemented

    def __radd__(self, other):          # 0 + Money, from sum()
        return self if other == 0 else NotImplemented

    def __mul__(self, k):
        if isinstance(k, (int, float, Decimal)):
            return Money(round(self.pence * k))
        return NotImplemented
    __rmul__ = __mul__

    def __lt__(self, other):
        return self.pence < other.pence

    def __str__(self):
        return f"£{self.pence / 100:.2f}"

fines = [Money(150), Money(275), Money(20)]
print(sum(fines), 3 * Money(150), max(fines), Money(1) >= Money(1))
```

← fills in <=, >, >= from < and ==

```
£4.45 £4.50 £2.75 True
```

```
Money(100) + 5
```

```
TypeError: unsupported operand type(s) for +: 'Money' and 'int'
```

RULE OF THUMB

Return `NotImplemented` (don't raise) when you don't recognise the other operand. Python then tries the reflected method and only raises `TypeError` if both sides give up, with a clear message. Money is stored in integer pence: never in a `float` (Ch 11).

Properties: attributes with rules

A `@property` looks like an attribute to the caller but runs code. Use it to **protect an invariant** (a rule that must always hold) or compute a value, without changing how callers write `obj.days`:

```
class Loan:
    MAX_DAYS = 28

    def __init__(self, title, days=14):
        self.title = title
        self.days = days                # goes through the setter below

    @property
    def days(self):
        return self._days

    @days.setter
    def days(self, value):
        if not 1 <= value <= self.MAX_DAYS:
            raise ValueError(f"days must be 1..{self.MAX_DAYS}, got {value}")
        self._days = value

    @property
    def weeks(self):                    # computed, read-only
        return self.days / 7

loan = Loan("Dune")
loan.days = 21
print(loan.days, loan.weeks)
loan.days = 90
```

```
21 3.0
ValueError: days must be 1..28, got 90
```

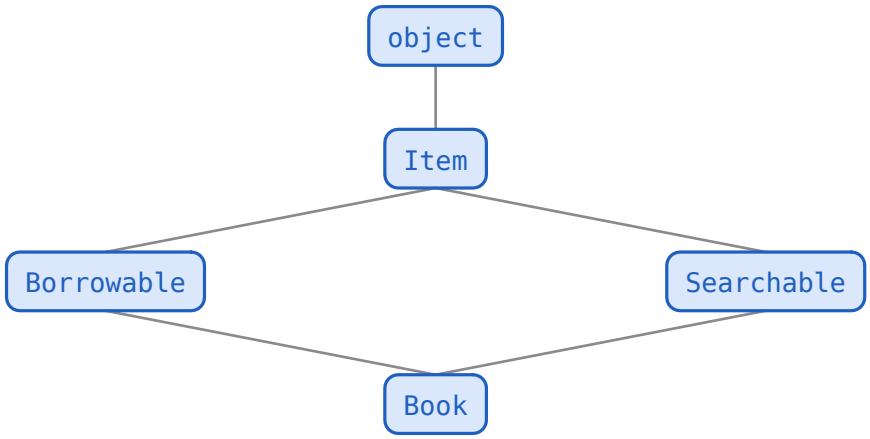
line 6: validated even in __init__

IN OTHER LANGUAGES

Java and C# write `getDays()`/`setDays()` up front, in case validation is needed later. Python starts with a plain attribute and swaps in a property only when needed, and callers don't change. The leading underscore in `_days` means "internal" by convention only.

Inheritance, composition and the MRO

Inheritance says "is a"; composition says "has a". Prefer composition unless the subclass truly can be used everywhere the base class can. When you do inherit, `super()` calls the next class in the **method resolution order** (MRO), a single line Python computes from the class tree:




```python
class Item:
    def describe(self):
        return ["item"]

class Borrowable(Item):
    def describe(self):
        return ["borrowable"] + super().describe()

class Searchable(Item):
    def describe(self):
        return ["searchable"] + super().describe()

class Book(Borrowable, Searchable):
    pass

print(" → ".join(c.__name__ for c in Book.__mro__))
print(Book().describe())
```

← next in the MRO, not 'my parent'

```
Book → Borrowable → Searchable → Item → object
['borrowable', 'searchable', 'item']
```

`super()` in `Borrowable` called `Searchable`, a class it has never heard of, because that's next in `Book`'s MRO. Every class runs exactly once, even in a diamond.

Interfaces: ABCs and duck typing

```python
from abc import ABC, abstractmethod
from typing import Protocol

class Notifier(ABC):                    # nominal: subclasses must implement send
    @abstractmethod
    def send(self, to, text): ...

class CanSend(Protocol):                # structural: anything with a send() matches
    def send(self, to, text): ...

class Email(Notifier):
    def send(self, to, text):
        return f"mail {to}: {text}"

print(Email().send("ana", "due"))
Notifier()
```

```
mail ana: due
TypeError: Can't instantiate abstract class Notifier without an implementation for abst…
```

Enums: a fixed set of named values

```python
from enum import Enum, auto

class Status(Enum):
    ON_SHELF = auto()
    ON_LOAN = auto()
    LOST = auto()

s = Status.ON_LOAN
print(s, s.name, s.value, s is Status["ON_LOAN"], [m.name for m in Status])
match s:
    case Status.ON_LOAN | Status.LOST:
        print("not available")
```

```
Status.ON_LOAN ON_LOAN 2 True ['ON_SHELF', 'ON_LOAN', 'LOST']
not available
```

RULE OF THUMB

Use an `Enum` instead of magic strings (`"on_loan"`) or numbers: typos become `AttributeError`s, editors autocomplete, and `match` reads clearly. `IntEnum`/`StrEnum` when the values must also behave like ints or strings (e.g. stored in SQLite).

need	reach for
a bag of fields with behaviour	a class; `@dataclass` if it's mostly data
an immutable record	`@dataclass(frozen=True)` or `typing.NamedTuple`
a fixed set of choices	`enum.Enum`
"anything with a `.send()`"	duck typing; `typing.Protocol` to document it

shared behaviour with enforced methods	`abc.ABC` with `@abstractmethod`

▢ Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 5.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 5.1` shows other valid solutions and their trade-offs.

`5.1` build from a spec A 2D vector that feels built in
`5.2` fix the bug Shared state and a vanishing set member
`5.3` build from a spec A container with slicing
`5.4` build from a spec Temperatures with rules
`5.5` learn an unseen module Who defines this method?
`5.6` mini-project Mini-project: a library catalogue

PASSPORT STAMP ask the same question of every new language (Appendix B)

7. Abstraction

Classes with single and multiple inheritance (C3 MRO, cooperative `super()`); duck typing via protocols of dunder methods; ABCs and `typing.Protocol` for explicit interfaces; properties for invariants; dataclasses for records; enums for choices.

6. Data structures

User types plug into built-in syntax by implementing protocols: `__len__`, `__getitem__`, `__iter__`, `__contains__`, `__eq__`/`__hash__`, arithmetic with `NotImplemented`.

DEEP DIVE (your vault)

Vault notes: 9.3 Instance attributes, class attributes, and lookup · 9.4 Methods are functions transformed by descriptor access · 9.7 Representation methods · 9.8 Equality, hashing, and immutable value objects · 9.10 Composition and delegation · 9.11 Polymorphism and protocols · 9.12 Special methods and operator protocols · 9.13 Cooperative binary operations and NotImplemented · 9.14 Multiple inheritance and MRO · 9.15 Descriptors · 9.17 Slots.